ChatGPT

Automatic Font Switching for English and Thai in Next.js with TailwindCSS

Problem Overview: Dual Fonts for Different Languages

Using **Bodoni Moda** for English (Latin script) and **Sukhumvit** for Thai on the same site requires an automatic font switch based on the text's language. The goal is to avoid manually wrapping text with special tags or classes in the HTML. We'll compare **pure CSS solutions** (leveraging Unicode ranges and the CSS `:lang()` selector) versus **JavaScript-enhanced approaches** (dynamic font loading or content detection). Our focus is on **performance**, **maintainability**, and **accuracy** of language detection, tailored for a Next.js + TailwindCSS setup with self-hosted font files.

CSS-Only Solutions for Language-Based Font Switching

1. Unicode-Range in `@font-face` (Recommended)

One of the most robust solutions is to use the `unicode-range` descriptor in CSS to **split font files by script**. This allows us to serve Bodoni Moda for Latin characters and Sukhumvit for Thai characters **under a single unified font-family**. The browser will automatically choose the appropriate font file for each character, with no manual markup needed.

How it works: We define two `@font-face` rules with the same font-family name but different Unicode ranges. For example:

```
/* In styles/globals.css or a Tailwind base layer */
@font-face {
  font-family: 'MultiLangFont'; /* Unified alias for both fonts */
  src: url('/fonts/BodoniModa-Regular.woff2') format('woff2');
  font-weight: 400;
  font-style: normal;
  unicode-range: U+0000-00FF, U+0100-024F; /* Latin + extended Latin range */
}
@font-face {
  font-family: 'MultiLangFont';
  src: url('/fonts/Sukhumvit-Regular.woff2') format('woff2');
  font-weight: 400;
  font-style: normal;
  unicode-range: U+0E00-0E7F; /* Thai Unicode block */
}

/* Apply the unified font-family globally */
html {
  font-family: 'MultiLangFont', serif; /* include a generic fallback */
}
```

In this setup, all text will use the **MultiLangFont** family. Under the hood, English letters (Unicode U+0000–024F) will be drawn using the Bodoni Moda file, and Thai characters (U+0E00–0E7F) will use the Sukhumvit file. This happens automatically per character. If a page contains only English text, the Thai font file is never downloaded, and vice versa – the browser only fetches a font file if at least one character on the page falls in its `unicode-range` [1]. This is great for performance, as each user only loads the fonts needed for the content they see. (For example, a site with separate English and Thai pages will only load the Bodoni font on English pages, and load Sukhumvit only on pages that contain Thai text [2].)

Benefits:
- *Automatic per-character font selection:* No need to tag or wrap text – the correct font is applied to each glyph by the browser. For example, if a paragraph is mostly English with a Thai word in the middle, the English letters render in Bodoni and the Thai letters in Sukhumvit seamlessly.
- *Performance:* Using `unicode-range` effectively subsets the fonts. Users don't pay the cost for unused character sets. If a page doesn't contain Thai text, the Thai font file won't be loaded at all [1]. If a page has both scripts, both font files will be fetched, but the browser can do this in parallel since it knows upfront which ranges are needed.
- *Maintainability:* The setup is declared once in CSS. Adding new content in either language doesn't require any code changes or additional markup – it will automatically use the correct font.
- *Accuracy:* This method is as accurate as Unicode can get – Thai characters and Latin characters are distinct code blocks, so there's no ambiguity. Every English letter will get Bodoni Moda and every Thai glyph will get Sukhumvit by definition.

Next.js + Tailwind Integration: In a Next.js project, you can include the above CSS in your global stylesheet (e.g. `globals.css` imported in your `_app.js` or root layout). Since these are custom fonts, ensure you have the font files in the `public/fonts` directory (or another accessible location). TailwindCSS will preserve these `@font-face` rules as long as the file is included.

You can then define a font-family in Tailwind's theme to use this combined family. For example, in your `tailwind.config.js`:

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      fontFamily: {
        // Define a Tailwind utility for the unified font
        'multi': ['MultiLangFont', 'serif']
      }
    }
  },
  // ...other config
};
```

Now `class="font-multi"` can be used to apply Bodoni-for-English and Sukhumvit-for-Thai. If you want it as the default font for your site, you could apply `font-multi` on the `<html>` or `<body>` element (using a custom `_document.js` in Next to add a class), or simply override the default sans/serif in Tailwind's base styles. For example, to globally apply it, you might add to your CSS:

```
@layer base {
  body {
    @apply font-multi;
  }
}
```

This ensures all text by default uses our combined font stack.

Note: Make sure to include all necessary Unicode ranges for your use case. In the example above, we included basic Latin and Latin-Extended letters for Bodoni. You may want to add punctuation or symbols to the appropriate font's range. (Common punctuation and numerals are usually in the basic Latin block which Bodoni covers. Thai punctuation and numerals fall in the Thai block with Sukhumvit.) The key is to avoid overlaps if possible – e.g., if Bodoni Moda has *some* glyphs for Thai (unlikely, but check), you might still restrict it to only Latin range to force Thai text into Sukhumvit. Similarly, if you prefer Thai context to use Thai-style digits, you could include the Western digit range in Sukhumvit's rule as well. Adjust the unicode ranges based on the glyph coverage of your font files.

2. Fallback Font Stack in CSS

A simpler (though slightly less fine-tuned) pure CSS approach is to leverage the natural **font-family fallback mechanism**. You can specify Bodoni Moda first and Sukhumvit second in a font stack. For example:

```
html {
  font-family: 'Bodoni Moda', 'Sukhumvit', serif;
}
```

With this configuration, the browser will attempt to render text with Bodoni Moda. For any character that Bodoni Moda **cannot display** (e.g. Thai characters, since Bodoni likely lacks them), the browser falls back to the next font in the list, which is Sukhumvit. In effect, English letters will use Bodoni, and Thai letters will use Sukhumvit, achieving the desired result [3]. As one Next.js user described a similar setup: when using `font-family: 'Poppins', 'NotoSansKR'` in CSS, *"'Hi there' will be Poppins and '안녕하세요' will be NotoSansKR"* (the Korean text fell back to the second font) [3]. The same principle applies here with Bodoni and Sukhumvit.

Pros: This requires no special @font-face unicode-range setup — you just load both fonts and list them. It also works per character automatically.

Cons: Compared to the unicode-range method, there are a few subtle drawbacks: - *Loading behavior:* Without unicode-range hints, the browser might not know upfront which font file is needed for which characters. It will load Bodoni Moda for sure (since it's first in the stack for essentially all text). If Thai characters are present, it will then load Sukhumvit when it encounters those glyphs are missing from Bodoni. This can lead to a slight delay in Thai text rendering (a flash or fallback to a default font until Sukhumvit is loaded) if the Thai font isn't already loaded. The unicode-range approach avoids this by indicating to the browser exactly which font covers Thai, so it can fetch it immediately.
- *Unused font loading:* Browsers may or may not delay loading the fallback font until needed — behavior can vary. There's a possibility that some browsers might preemptively load the second font if it's listed, but generally they try to be efficient. In either case, if your page has no Thai text, Sukhumvit **might not**

be fetched. But to be safe with performance, unicode-range is more explicit about not downloading unless needed.
- *Mixed context styling:* With pure fallback, the font choice is purely per glyph. This means if, for example, an English sentence contains a single Thai character (like a Thai currency symbol or a loanword), that one character will flip to Sukhumvit while the rest is Bodoni. This achieves the technical goal, but the visual mix of a sans-serif symbol in a serif sentence might be noticeable. In many cases this is fine, or even desirable for clarity. However, if you preferred to keep a consistent style *per language context* (say, even Latin numerals in a Thai paragraph should use the Thai font to maintain a uniform look), the fallback mechanism can't enforce that – it will use Bodoni for any Latin-range character it sees, even if it's within a Thai sentence. (Solving that would require marking up that context or using the method below with `:lang` selectors.)

In practice, the fallback method often works well for distinct scripts like Latin vs Thai. It is certainly easier to set up: just include both fonts via `@font-face` (without unicode-range) and list them in the desired order in your Tailwind config or CSS. For example, you could do in `tailwind.config.js`:

```
extend: {
  fontFamily: {
    sans: ['Bodoni Moda', 'Sukhumvit', 'serif']
  }
}
```

Then use `font-sans` on your elements (or let Tailwind's base `<body>` use it by default). Ensure Bodoni is listed first so that English (Latin) uses Bodoni and falls back to Sukhumvit for Thai. This approach meets the "no hardcoding in HTML" requirement, since the browser handles the switch automatically for each character. Just be aware of the above caveats. For maximum performance and control, you can combine this with `font-display: swap` in your @font-face rules to prevent blocking text rendering, and use `preconnect` or `preload` hints if needed – although note that preloading a font can bypass unicode-range optimizations (the font will load regardless of content) [4].

3. CSS `:lang()` Selector with Language Attributes

Another pure CSS method is to use the `:lang()` pseudo-class to apply fonts based on the language of an element's text. For example:

```
*:lang(en) {
  font-family: 'Bodoni Moda', serif;
}
*:lang(th) {
  font-family: 'Sukhumvit', sans-serif;
}
```

This CSS will target any element declared to be English or Thai and assign the respective font. The benefit is that it switches the font on a *per-element* basis, so you could ensure, for instance, an entire `<p lang="th">...</p>` uses Sukhumvit for all characters (including any Latin letters or numbers in that paragraph). Similarly an English-marked element would use Bodoni for everything, including punctuation or the occasional Thai character (though mixing Thai chars in an English paragraph is rare). This approach can provide a more uniform typographic style within a given language context.

However, using `:lang()` **requires your HTML content to be properly annotated** with `lang` attributes (or inherited language from a parent). The browser doesn't magically detect language from text content for this selector—you must declare it. For example, you would wrap Thai text as `<span lang="th">สวัสดี</span>` inside an English page, or at least set `<html lang="en">` on English pages and `<html lang="th">` on Thai pages. If you already have a multi-lingual site, you *should* be using these attributes for accessibility and SEO anyway. The `:lang` pseudo-class will match elements based on those attributes (and it even inherits through the DOM, so a child element without a lang will be treated as the same lang as its nearest ancestor that has one [5] [6]).

Example integration (Tailwind): You can incorporate this into Tailwind by extending the base styles or using a plugin/variant for `:lang`. A straightforward way is adding to your global CSS (Tailwind can preserve it via `@layer base`):

```
@layer base {
  *:lang(en) {
    @apply font-bodoni;   /* assuming you've defined font-bodoni in theme */
  }
  *:lang(th) {
    @apply font-sukhumvit; /* font-sukhumvit defined in theme extend */
  }
}
```

Where your Tailwind config might include:

```
extend: {
  fontFamily: {
    'bodoni': ['Bodoni Moda', 'serif'],
    'sukhumvit': ['Sukhumvit', 'sans-serif']
  }
}
```

Now any element like `<p lang="th">...</p>` will get the Sukhumvit font via the above CSS. (If you prefer, you could target specifically elements like `html:lang(th) body { ... }` to apply a font globally when the page language is Thai, as was suggested in a Tailwind context [7]. But that covers one language per page rather than mixed content on one page.)

Pros:
- *Clear intent:* You explicitly control which sections of the DOM use which font. This can prevent odd mismatches (like the digit style issue mentioned earlier, since an entire Thai paragraph marked lang="th" would use Sukhumvit for everything, including numbers or Latin letters in it).
- *Leverages standards:* This method uses standard HTML language tags, which is good for accessibility. Screen readers, search engines, and translation tools benefit from proper `lang` attributes.

Cons:
- *Requires markup changes:* The requirement "no hardcoding in HTML" suggests the user might not want to manually sprinkle lang attributes or extra spans in their content. If your content source or CMS already gives you structured language info, great – but if not, adding these attributes could be labor-intensive or not feasible.

- *Maintenance:* You must remember to mark any new content with the correct `lang`. Missing an attribute on mixed-language content means the styling won't apply correctly.
- *Granularity:* If your content frequently mixes languages within the same sentence or even word, wrapping each switch with a span is tedious. (Though mixing Thai and English usually happens at phrase boundaries, so a span wrap might be manageable.)

In summary, `:lang()` CSS is *ideal when you have well-defined language blocks*, such as entire pages or large elements known to be one language or another. In fact, the W3C recommends `:lang` as the most appropriate way to style content by language [8]. It's very maintainable in scenarios like switching global font families on a locale basis (similar to how one might do for Arabic vs Latin script globally). But for sporadic, unpredictable mixing of scripts, it may not be as "automatic" unless the source is annotated or detected somehow.

Which CSS Approach to Choose?

For the scenario described (English and Thai text intermixed on the site, with no manual markup for switching), the **Unicode-range method (Approach #1)** is arguably the best fit. It gives you automatic, per-character font assignment with zero runtime overhead and no need for additional attributes. It's essentially "set and forget": once configured, any text that falls in Thai Unicode range will render in the Thai font. Performance is also excellent because of on-demand font loading [1].

Using `:lang` selectors (#3) is powerful if you are willing to maintain language attributes in your HTML (or if your content is already structured by language). It might be overkill for just two scripts that are easily distinguishable by Unicode. But if in the future you add more languages or need more nuanced control (say, a different font for numbers or a third language), a strategy involving `:lang` could scale better.

The fallback stack approach (#2) is a quick solution that works out-of-the-box and might be sufficient for many cases. It's probably the easiest to implement via Tailwind config. Just be mindful of the slight performance hit of loading the second font when needed (which, if both scripts appear together, you'll incur anyway) and potential flash if the second font kicks in late. In practice, using `font-display: swap` in your @font-face for Sukhumvit can mitigate flashes (the Thai text might show in a system font for a brief moment and then swap to Sukhumvit once loaded).

Tip: Whichever CSS route you choose, ensure your font files are optimized. Use **WOFF2** format for best compression, and consider subsetting the font files to only include the needed characters for each (which unicode-range already hints at) [9] [10]. Since you are self-hosting, you have control to run a font subsetter tool to strip out glyphs you'll never use (e.g., if Bodoni Moda file contains Cyrillic, you can remove those to reduce size, given you only need Latin; similarly remove Latin from Sukhumvit if it contains some). Smaller font files = faster loading.

JavaScript-Enhanced Approaches

While pure CSS can handle this use-case well, there are some scenarios where JavaScript might come into play:

1. Using Next.js `next/font` for Conditional Loading

Next.js 13+ introduced the `next/font` utility, which helps with self-hosting fonts and optimizing loading. You can use it to load your Bodoni Moda and Sukhumvit fonts as **separate font families** and

then conditionally apply them. For example, in `app/layout.tsx` (or `_app.js` for older Next versions):

```
// next/font for local fonts
import localFont from 'next/font/local';

// Define Bodoni Moda font (Latin only)
const bodoni = localFont({
  src: [
    { path: '../public/fonts/BodoniModa-Regular.woff2', weight: '400' },
    { path: '../public/fonts/BodoniModa-Bold.woff2', weight: '700' }
  ],
  display: 'swap',
  variable: '--font-bodoni'  // use CSS variable for Tailwind
});

// Define Sukhumvit font (Thai)
const sukhumvit = localFont({
  src: '../public/fonts/Sukhumvit-Regular.woff2',
  display: 'swap',
  variable: '--font-sukhumvit'
});

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={`${bodoni.variable} ${sukhumvit.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

This will auto-generate the necessary @font-face in your CSS, and embed CSS variables `--font-bodoni` and `--font-sukhumvit` that reference those font-families. We include both `.variable` classNames on `<html>` so that both fonts are loaded and their variables available. Now in Tailwind, you can set up a combined font-family using those variables:

```
// tailwind.config.js
extend: {
  fontFamily: {
    'english-thai': ["var(--font-bodoni)", "var(--font-sukhumvit)", "serif"]
  }
}
```

Applying `font-english-thai` to an element (or the body) will result in a CSS `font-family: BodoniModa, Sukhumvit, serif` effectively. This is similar to the fallback approach but using Next's font loader. If your site typically uses one language at a time (e.g., you have distinct English vs Thai pages), you could optimize by *not* loading both fonts together. Instead, you might import only Bodoni in the layout for English pages and only Sukhumvit for Thai pages (perhaps via separate layouts or a

runtime check on locale). This way, an English-only page doesn't even include Sukhumvit, saving that download altogether. Next.js makes it easy to have per-page or per-route font definitions.

However, if your pages truly mix the languages or you want both fonts available on the same page, you will be loading both regardless. The above approach with `next/font` is basically a convenience for self-hosting and eliminating FOUT/CLS (it inlines the font CSS and can preload it). It doesn't inherently do "language detection" – you still rely on CSS fallback or unicode-range logic for actual switching.

Note: As of Next.js 13, `next/font` (particularly the Google font loader) doesn't directly support a `unicodeRange` option for splitting fonts by script [11] [12]. The recommended workaround from Vercel for multi-script scenarios is to disable `adjustFontFallback` on one of the fonts and include both, then use CSS to define the fallback order [13] [14]. In simpler terms, the approach we showed above (loading both and combining via CSS) is the way to go. The Next/font system will ensure the fonts are subsetted to the specified characters (e.g., if you use Google Fonts with subsets), but for local fonts you handle that yourself.

2. Dynamic Language Detection on the Client

Another possible JS approach is to use client-side logic to identify which portions of the text are Thai vs English and then wrap or style them. This could be done by scanning the DOM text nodes after render, or by using a library that detects scripts. For example, one could write a script to find any Thai Unicode characters (regex like `/[\u0E00-\u0E7F]/`) and then wrap those words in a `<span class="font-sukhumvit">` or add a class to the parent element. However, this approach is generally **not recommended** unless you have a very dynamic content scenario that can't be handled by static CSS. It introduces complexity and can impact performance (especially if you have to iterate over potentially large text content). There's also a flash risk: content might render in the wrong font and then snap to the new font once your JS runs. Maintaining such a script for accuracy (e.g., ensuring it doesn't mis-tag something or run at the right time) would be an extra burden.

If your use-case is something like **user-generated content** where any mix of languages could appear and you absolutely cannot preprocess or mark it up, a JS solution could be a fallback. But even then, a better solution might be server-side processing – e.g., detect the language of a block of text server-side (using a library or heuristic) and output a span or a `lang` attribute accordingly. That way the decision is made once and CSS can handle the rest.

3. Font Loading Strategies

Another JavaScript-enhanced angle is performance tuning with font loading. For instance, you might deliberately defer loading the less-used font until it's needed. You could do this by not including the @font-face for Sukhumvit initially, and then using the **Font Loading API** or a library like FontFaceObserver to load it on the fly when needed (say, if the user switches language or if you detect Thai content became visible). Next.js `next/font` currently doesn't support truly lazy-loading a font after first paint – it's more for initial load optimization. If you needed that, you'd have to roll a custom solution (e.g., a `<link rel="stylesheet">` to a font CSS injected on demand).

However, given that Thai text is likely essential content (not something you want to delay or hide), lazy-loading that font could harm user experience for Thai readers. So this kind of micro-optimization only makes sense if Thai text is extremely rare or non-critical on some pages. In most cases, you should prioritize loading both fonts early when both are needed.

Summary of JS vs CSS: Using JavaScript for this problem is usually only justified for conditional loading (to avoid unnecessary font downloads) or if your content structure is too dynamic to handle otherwise. In a Next.js app with defined locales or content, you can often achieve conditional font inclusion without heavy scripting: e.g., load Bodoni font in the English locale routes and Sukhumvit in the Thai routes (thus each page only ships one font). If English and Thai are on the same page, then you'll ship both fonts no matter what – so a CSS solution that gracefully handles the mix is ideal. The CSS-only approaches have the advantage of running at the browser's native level (very fast and no script needed). They are easy to maintain (the rules don't typically need frequent changes), whereas a JS solution might break if the DOM structure changes or require updates if a new language is added.

From an **accuracy** standpoint: The Unicode-range method is basically foolproof for distinguishing Latin vs Thai. The `:lang` method is also very accurate *if* your content is correctly labeled (the heavy lifting of identifying language is on you). A JS language-detector might misidentify short strings or transliterated text, etc., so it's less reliable than simply using the inherent Unicode script property or explicit metadata.

Recommendation and Code Samples

For a Next.js + TailwindCSS project with self-hosted fonts, here is a recommended setup combining the discussed ideas:

- **Font Hosting:** Place your font files in the `/public/fonts` folder. Use WOFF2 format if possible for smaller size [15]. Ensure you have Bodoni Moda (likely only Latin chars needed) and Sukhumvit (Thai chars).

- **Global CSS with @font-face:** In your `globals.css` (or equivalent), define the font faces with unicode ranges as shown in Approach #1. Use a common `font-family` name (like `"MultiLangFont"` in our example) or simply plan to use both font names in a stack. For clarity, let's use separate names and a stack in Tailwind:

```
/* styles/fonts.css - imported in globals.css or directly in _app.tsx */
@font-face {
  font-family: 'Bodoni Moda';
  src: url('/fonts/BodoniModa-Regular.woff2') format('woff2');
  font-weight: 400;
  font-style: normal;
  unicode-range: U+0000-007F, U+0100-024F, U+2000-206F; /* Latin basic +
extended + punctuation */
}
@font-face {
  font-family: 'Bodoni Moda'; /* include bold, italic, etc as needed with
same name & appropriate unicode-range */
  src: url('/fonts/BodoniModa-Bold.woff2') format('woff2');
  font-weight: 700;
  font-style: normal;
  unicode-range: U+0000-007F, U+0100-024F, U+2000-206F;
}
@font-face {
  font-family: 'Sukhumvit';
  src: url('/fonts/Sukhumvit-Regular.woff2') format('woff2');
```

```
  font-weight: 400;
  font-style: normal;
  unicode-range: U+0E00-0E7F; /* Thai characters */
}
/* You can add font-display: swap; to each @font-face if desired */
```

(Ensure the unicode-range covers all needed glyphs. The above includes general punctuation in Bodoni; you might also include digits 0-9 in both ranges depending on design preference.)

- **Tailwind Configuration:** Extend Tailwind to include these fonts. For example, in `tailwind.config.js`:

```
module.exports = {
  theme: {
    extend: {
      fontFamily: {
        english: ['Bodoni Moda', 'serif'],
        thai: ['Sukhumvit', 'sans-serif'],
        // Or a combined stack:
        dual: ['Bodoni Moda', 'Sukhumvit', 'serif']
      }
    }
  },
  // ...
};
```

You have a few options here:

- Use a **combined stack** (`font-dual`) globally. This is simplest: apply `font-dual` on the `<html>` or `<body>` and let browser do per-char fallback. This will leverage the unicode-range optimizations we set (only load Sukhumvit if Thai chars are present, etc.).
- Or, use conditional classes: `font-english` and `font-thai` could be applied via the `:lang` selector approach. For example, add in your CSS:

```
@layer base {
  *:lang(th) { @apply font-thai; }
  *:lang(en) { @apply font-english; }
}
```

and ensure elements (even `<html>`) have `lang` attributes. This way an English page (<html lang="en">) will use Bodoni by default, and a Thai page (<html lang="th">) will use Sukhumvit, but also an English snippet inside a Thai page (marked lang="en") would switch to Bodoni appropriately [16] [7]. This gives very fine-grained control if needed.

If you prefer not to rely on lang attributes at all, then the combined approach (`font-dual`) is the way to go – it uses the unicode-range mechanism internally without any markup.

- **Applying the Font:** Finally, apply the font to your site. If you used the combined family (font-dual), you could do:

```
// _app.js or layout.jsx
import '@/styles/
fonts.css';  // ensure your @font-face declarations are loaded
import '@/styles/globals.css';
// ...
function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} className="font-dual" />;
}
```

Or better, in `globals.css` use:

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html {
    @apply font-dual;
  }
}
```

This sets the default font for the entire app to our dual stack. All text will automatically pick the appropriate font as it renders.

Testing: After implementing, test pages that have English-only, Thai-only, and mixed content. Use your browser's dev tools network panel to verify font loading: on an English-only page, the Sukhumvit font file should not download; on a Thai page, both might download (if there are any ASCII characters, Bodoni will load for those, unless Sukhumvit also covers basic Latin as a fallback). Also inspect the rendered text to ensure the switch happens correctly (e.g., DevTools computed styles can show which font-family is applied to a given element or even which font file a specific glyph came from).

Pros and Cons Recap

CSS Approach (unicode-range or fallback): - **Pros:** No runtime overhead, straightforward to implement, leverages browser's optimized behavior. Very accurate per character. Maintainable (just a config in CSS). Using unicode-range provides automatic font subsetting by language [2]. Using `:lang` can enhance semantic correctness. - **Cons:** Need to be careful with range definitions and font coverage. Fallback method might cause slight flashes if secondary font loads late (mitigated by `swap` and quick loading). `:lang` approach requires adding `lang` attributes in HTML, which some might consider "hardcoding" (though it's semantic data rather than presentational).

JS/Next.js Approach: - **Pros:** With Next's font optimizations, easy to self-host and avoid layout shifts. Possibility of conditionally including fonts only when needed (improves initial load for single-language pages). Can handle complex logic (if needed) to dynamically apply classes or font loading. - **Cons:** Added complexity isn't necessary for a problem already solvable by CSS. As seen in community discussions, Next's font system doesn't natively handle "two fonts in one element" without manual steps [17] [18]. You often end up implementing the CSS fallback anyway (for example, developers resorted to wrapping different languages in `<span>` with different classes when using next/font [19] [20] – essentially re-

introducing the manual markup we wanted to avoid). A client-side language detection script is brittle and can slow down rendering.

Maintainability: The CSS solution will adapt automatically to new content. If you later add a third language (say Japanese with its own font), you can extend the same pattern: add a @font-face with that Unicode range and include it in the font-family stack or another `:lang()` rule. The JS solution would likely need code changes to handle the new language, increasing maintenance surface.

Conclusion

For a Next.js + TailwindCSS site needing **Bodoni Moda for English** and **Sukhumvit for Thai**, the best practice is to use a **CSS-based font switch**. Specifically, define the fonts with appropriate Unicode ranges and use a combined font-family or CSS fallback so the browser automatically picks the correct font for each character. This approach is performant (loads only what's needed) and requires no manual tagging of content. It keeps your HTML clean and your typography correct.

You can integrate this cleanly with Tailwind by defining custom font families and applying them globally. Should you require more explicit control, using `lang` attributes with the `:lang()` CSS selector is a standards-based solution that Tailwind can accommodate (with base layer styles or a plugin) [16] [7]. JavaScript methods, on the other hand, offer more dynamic control but usually aren't necessary here – the built-in browser capabilities are sufficient and more efficient for language-based font selection.

By following the outlined setup, you'll achieve a seamless experience: English text rendering in the elegant Bodoni Moda, Thai text in the appropriate Sukhumvit font, with the switch happening automatically under the hood. Users will simply see the right font for each language, and you won't need to hardcode anything in your content.

Sources:

- MDN Web Docs – *unicode-range*: Explanation of how specifying Unicode ranges in `@font-face` can split fonts by language and avoid unnecessary downloads [1].
- W3C I18N Techniques – *Styling by language*: Recommends using the `:lang()` CSS selector to target content in a given language for font-family changes [8].
- Stack Overflow (Janoma's answer) – Demonstrates using Tailwind's base layer with `*:lang(...)` to apply different font families for Arabic vs Latin, requiring proper `lang` attributes on HTML [7].
- Next.js community discussion – Confirms that setting a CSS font stack like `font-family: 'Poppins', 'Noto Sans KR'` results in English text using the first font and Korean text using the second via fallback [3]. This analogous behavior applies to Bodoni vs Sukhumvit.

---

[1] [2] unicode-range - CSS | MDN
https://developer.mozilla.org/en-US/docs/Web/CSS/@font-face/unicode-range

[3] [17] [18] [19] [20] multiple font and priority (next/font/google) : r/nextjs
https://www.reddit.com/r/nextjs/comments/11unuhk/multiple_font_and_priority_nextfontgoogle/

[4] [9] [10] [15] Best practices for fonts | Articles | web.dev
https://web.dev/articles/font-best-practices

5 6 8 Styling using language attributes
https://www.w3.org/International/questions/qa-css-lang/1000

7 16 css - How to change the default font family when locale changes using tailwind&nuxtjs? - Stack Overflow
https://stackoverflow.com/questions/74944966/how-to-change-the-default-font-family-when-locale-changes-using-tailwindnuxtjs

11 12 13 14 any plan to support `unicode-range` on `next/font/google`? · vercel next.js · Discussion #47309 · GitHub
https://github.com/vercel/next.js/discussions/47309